

SISTEMAS OPERATIVOS

3004610 - 1

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Hilos de ejecución
pthreads

Contenido

1. Procesos ligeros: concepto, ventajas, modelos de implementación, estados y paralelismo
2. La librería pthread: creación, terminación y espera de hilos; compilación
3. Ejemplos: creación de hilos y paso de argumentos



Procesos ligeros (hilos de ejecución)

Procesos ligeros: hilos de ejecución

- Un **proceso ligero** (*thread*, hebra o hilo) es un flujo de ejecución dentro de un proceso: usa los recursos del proceso y comparte con los demás hilos la imagen de memoria y otra información.
- Cada hilo tiene su propio **contador de programa, registros y pila**, porque ejecuta su propia secuencia de instrucciones.
- Un proceso tradicional (**monohilo**) tiene un único flujo de control; un proceso **multihilo** realiza varias tareas a la vez sobre el mismo espacio de direcciones.
- Los hilos de un mismo proceso comparten **código, datos y archivos abiertos**.

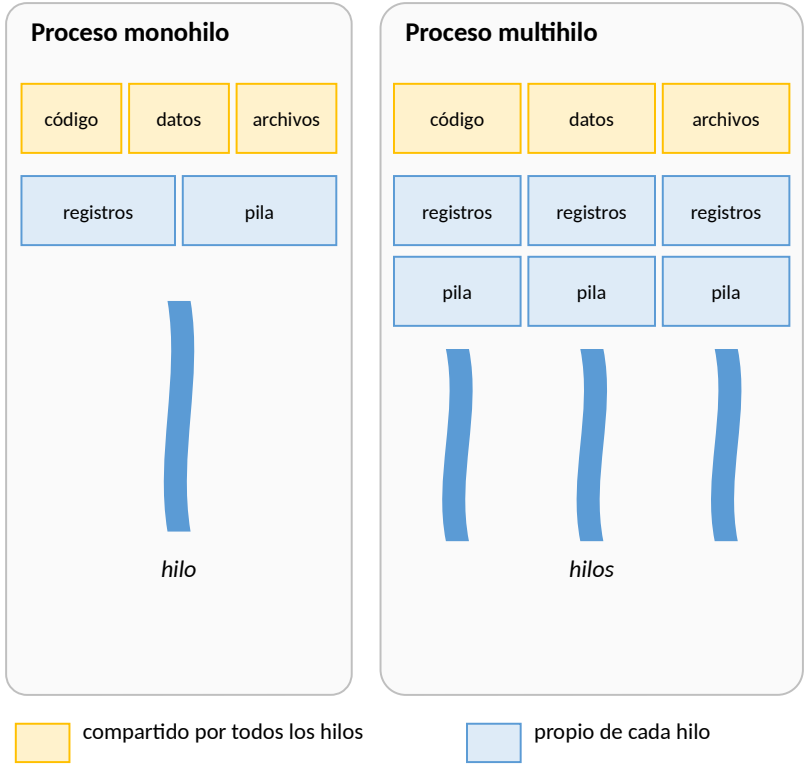
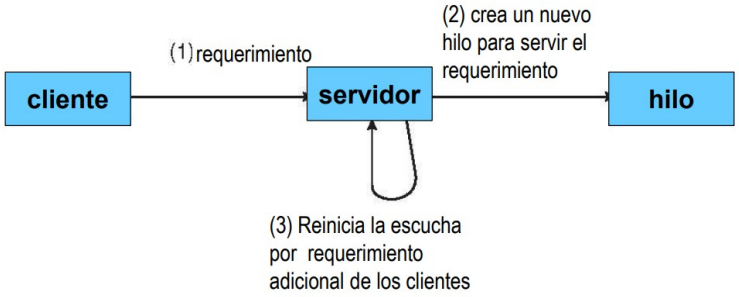


Diagrama adaptado de Silberschatz, Galvin y Gagne, Operating System Concepts, fig. 4.1.

Ventajas del modelo de múltiples hilos

- **Recursos compartidos:** los hilos de un proceso se comunican a través de la memoria común, sin intervención del kernel (no necesitan IPC).
- **Administración eficiente:** crear un hilo, cambiar de contexto entre hilos y terminarlos cuesta mucho menos que hacerlo con procesos.
- **Aprovechamiento de las arquitecturas multiprocesador:** los hilos de un proceso pueden ejecutarse en paralelo en varios núcleos.
- **Capacidad de respuesta:** un servidor atiende cada petición en un hilo nuevo mientras el hilo principal sigue escuchando (figura).



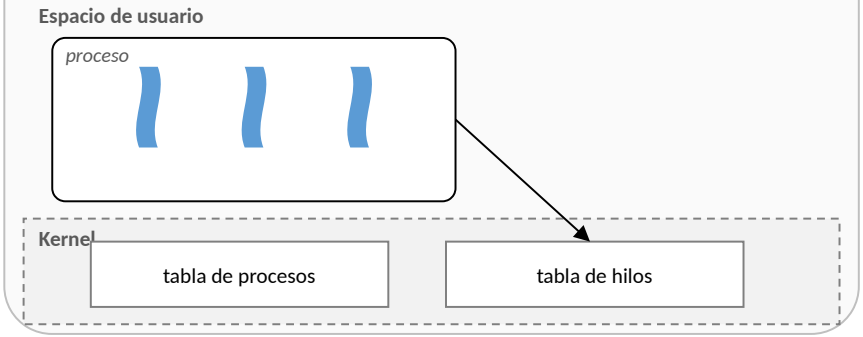
	Procesos	Hilos
Creación	`fork()`: copia el espacio de direcciones	`pthread_create()`: solo pila y registros
Comunicación	IPC: tuberías, colas, memoria compartida	Variables globales del proceso
Cambio de contexto	Costoso: cambia el mapa de memoria	Ligero: solo registros y pila

Servidor multihilo. Fuente: Silberschatz, Galvin y Gagne, Operating System Concepts, fig. 4.2 (traducida).

Modelo de hilos en el sistema operativo

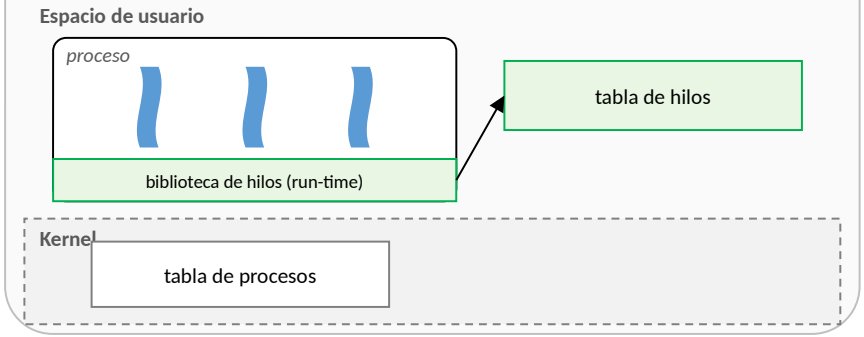
Hilos de nivel de kernel (KLT, kernel-level threads)

- Los administra directamente el sistema operativo: el kernel conoce y planifica todos los hilos.
- Una entrada en el bloque de control de proceso (PCB) por proceso y un bloque de control de hilo (TCB) por hilo.
- El SO ofrece llamadas al sistema específicas para crear y administrar hilos.
- Si un hilo se bloquea, los demás siguen; pueden ejecutarse en paralelo en varios núcleos.



Hilos de nivel de usuario (ULT, user-level threads)

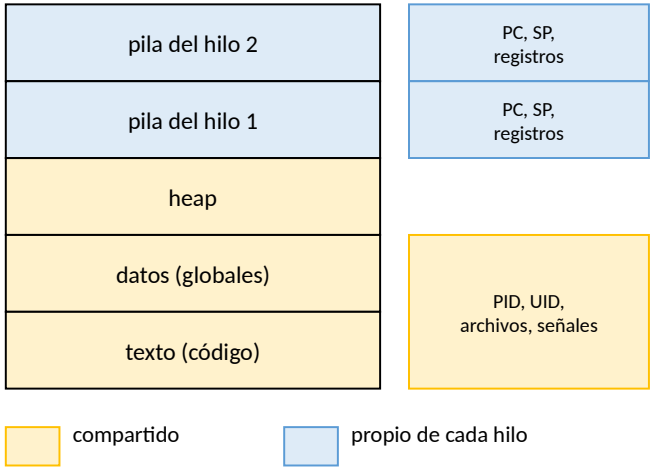
- Los administra una biblioteca en espacio de usuario, sin soporte del kernel del SO.
- El kernel no conoce los hilos: los planifica como si fueran un único proceso.
- Si un hilo hace una llamada al sistema bloqueante, el kernel bloquea todo el proceso; tampoco aprovechan varios núcleos.
- No requieren modificar el SO; crear hilos y cambiar entre ellos es muy rápido.



Información compartida y no compartida

Compartida por todos los hilos del proceso	Propia de cada hilo
• Espacio de memoria (código, datos, heap) • Variables globales • Archivos abiertos • Procesos hijos • Temporizadores • Manejadores de señales y semáforos • Contabilidad (tiempo de CPU consumido)	• Contador de programa • Pila (variables locales y llamadas) • Registros del procesador • Estado del proceso ligero (listo, ejecutando, bloqueado) • Máscara de señales y `errno`

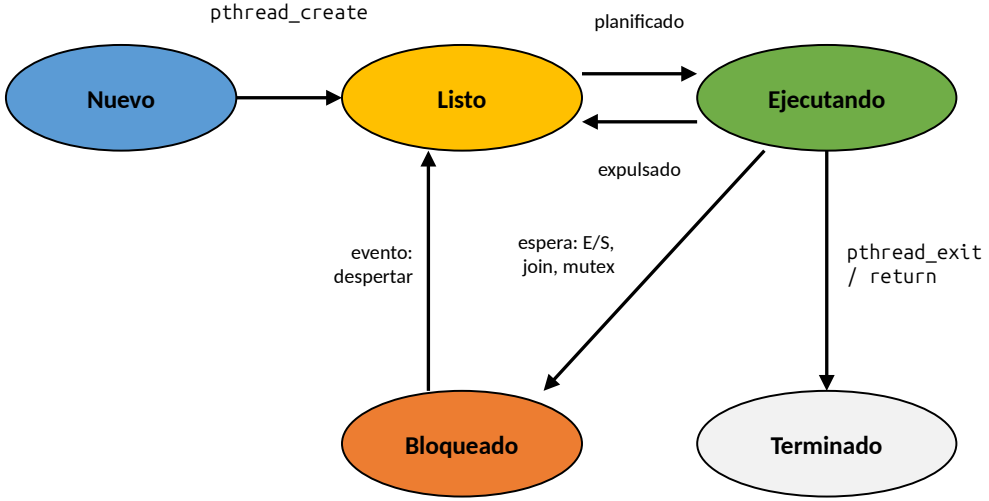
Espacio de direcciones del proceso

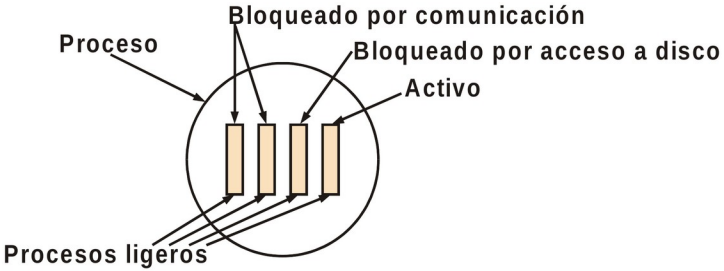


Los hilos se comunican simplemente leyendo y escribiendo las variables globales del proceso; a cambio, deben **sincronizar** sus accesos a esos datos (tema siguiente).

Estados de los hilos

- Un proceso ligero puede estar **ejecutando**, **listo** o **bloqueado**, igual que un proceso; el estado es propio de cada hilo.
- En un mismo proceso pueden coexistir hilos activos y bloqueados: mientras uno espera por E/S, otro sigue ejecutando.

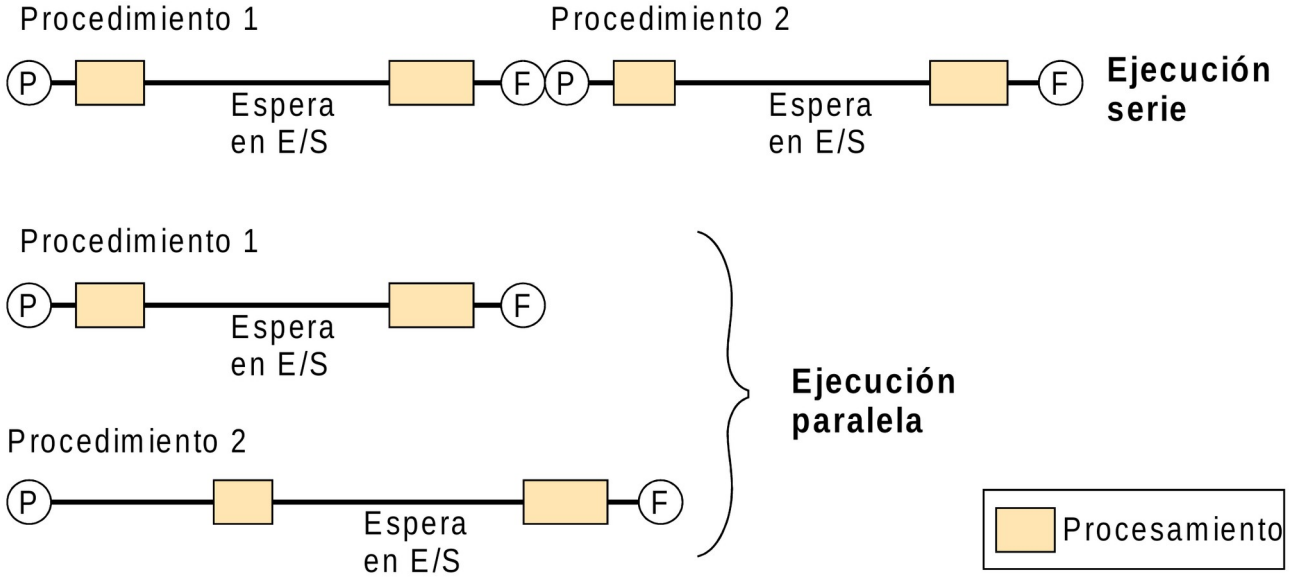




Un proceso con cuatro procesos ligeros en distintos estados. Fuente: Carretero, García, de Miguel y Pérez, Sistemas operativos: una visión aplicada.

Paralelismo: ejecución en serie y en paralelo

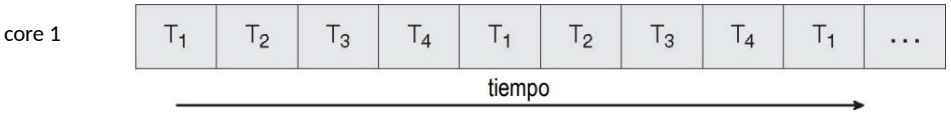
- Los procesos ligeros permiten **paralelizar una aplicación**: cada procedimiento se ejecuta en un hilo distinto.
- Mientras un hilo espera en E/S, otro hilo del mismo proceso puede seguir procesando: el tiempo total se reduce.



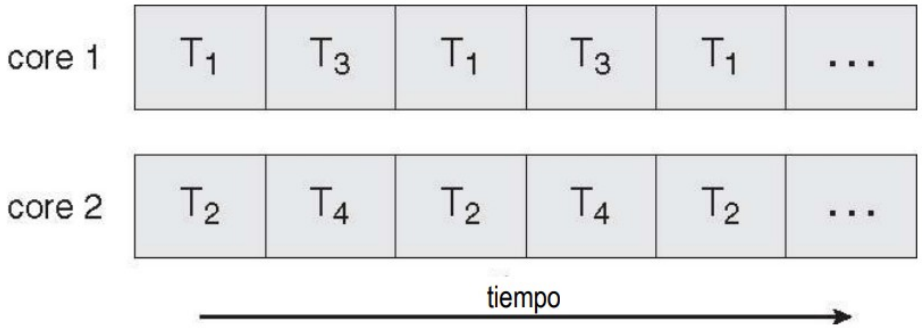
Fuente: Carretero, García, de Miguel y Pérez, Sistemas operativos: una visión aplicada.

Paralelismo: un núcleo frente a varios núcleos

Un solo núcleo: concurrencia (los hilos se alternan en el tiempo)



Varios núcleos: paralelismo (los hilos se ejecutan a la vez)



Fuente: Silberschatz, Galvin y Gagne, Operating System Concepts, figs. 4.3 y 4.4.

- **Concurrencia**: varios hilos progresan alternándose en un solo núcleo; en cada instante solo uno ejecuta.
- **Paralelismo**: varios hilos ejecutan simultáneamente en núcleos distintos.
- Con un solo núcleo los hilos siguen siendo útiles: solapan cómputo con esperas de E/S y simplifican el diseño de los programas.
- Con varios núcleos, un programa multihilo puede reducir su tiempo de ejecución casi en proporción al número de núcleos.



La librería pthread (hilos POSIX)

Pthread: hilos POSIX

- Es la implementación estándar de hilos en los sistemas derivados de Unix: **IEEE POSIX 1003.1c** (1995).
- API en lenguaje C para la creación, gestión y sincronización de hilos; tipos y funciones declarados en `<pthread.h>`.
- Los hilos pthread son de dos tipos: **acoplados** (*joinable*) y **desacoplados** (*detached*). Por defecto son acoplados.
- **Acoplado:** sus recursos se conservan hasta que otro hilo lo espera con `pthread_join`.
- **Desacoplado:** no se le puede esperar; el sistema reclama sus recursos al terminar. Un hilo desacoplado no puede volver a ser acoplado.

Funciones de pthread en este curso

Función	Uso
`pthread_create`	crear un hilo
`pthread_join`	esperar a que termine un hilo
`pthread_exit`	terminar el hilo que llama
`pthread_self`	identificador del hilo que llama
`pthread_cancel`	cancelar otro hilo
`pthread_mutex_*`	exclusión mutua (tema 12)

Creación de hilos: *pthread_create*

```
int pthread_create( pthread_t *tid , const pthread_attr_t *attr , void *(*function)(void *) , void *arg );
```

tid
Descriptor del hilo creado: pthread_create escribe aquí su identificador (parámetro de salida).

attr
Atributos del nuevo hilo (tamaño de pila, acoplado o no...). NULL: atributos por defecto.

function
Función que ejecutará el hilo: recibe un void * y devuelve un void *.

arg
Único argumento que se pasa a la función; NULL si no hay argumentos.

- Inicia un nuevo hilo que ejecuta `function(arg)` de forma concurrente con el hilo que lo creó; ambos comparten la memoria del proceso.
- Para pasar varios datos se envía un puntero a una estructura (`struct`); para no pasar nada, `NULL`.

Retorna 0 si tiene éxito. Si falla devuelve el código de error (no usa `errno`): **`EAGAIN`** recursos insuficientes para crear el hilo; **`EINVAL`** atributo inválido. **¡Siempre verificar el valor de retorno!**

Ficha de *pthread_create*

int pthread_create(tid, attr, function, arg);

Crea un nuevo hilo dentro del proceso que empieza a ejecutar la función `function` con el argumento `arg`.

Include	`<pthread.h>` (compilar con `gcc -pthread`)
Retorna	**Éxito:** 0; el identificador del nuevo hilo queda en `*tid`. **Falla:** un código de error (no usa `errno`): `EAGAIN` recursos insuficientes para crear el hilo; `EINVAL` atributo inválido.
Argumentos	• ***pthread_t *tid***: descriptor del hilo creado. • ***const pthread_attr_t *attr***: atributos del hilo; `NULL` = atributos por defecto (acoplado, pila estándar). • ***void *(*function)(void *)***: función que ejecutará el hilo. • ***void *arg***: argumento único para la función (`NULL` si no hay).
Nota	El hilo nuevo puede empezar a ejecutar antes de que `pthread_create` retorne: no hay ningún orden garantizado entre los hilos.

Terminación de un hilo

El hilo termina de una de estas maneras:

- Llama a **`pthread_exit(valor)`**: el valor de salida queda disponible para el hilo del mismo proceso que lo espere con `pthread_join`.
- Retorna de **`function()`**: equivale a llamar a `pthread_exit` con el valor de la sentencia `return`.
- Es cancelado por otro hilo con **`pthread_cancel(tid)`**.
- Termina el proceso: cualquier hilo llama a **`exit()`** o el hilo principal retorna de **`main()`**. Esto termina **todos** los hilos del proceso.

Las dos formas equivalentes de devolver un valor:

```
void *f(void *arg) {
    /* ... trabajo ... */
    pthread_exit(res);  /* o bien: */
    return res;
}

/* en main(): recoge el valor */
pthread_join(tid, &res);
```

Error común: el hilo principal retorna de `main()` sin esperar a los demás hilos: el proceso termina y los hilos mueren a medias. Espere con `pthread_join` (o termine `main` con `pthread_exit`, que deja vivos a los otros hilos).

Esperar a un hilo: *pthread_join*

```
int pthread_join(pthread_t tid, void **val_ptr);
```

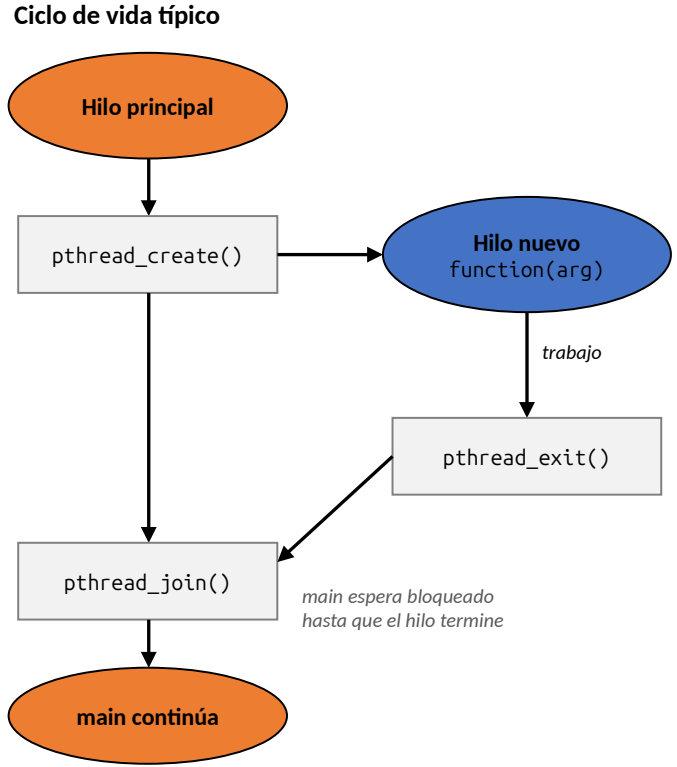
El hilo que llama se bloquea hasta que el hilo `tid` termina, y recoge su valor de salida.

Include	`<pthread.h>`
Retorna	**Éxito:** 0. **Falla:** un código de error: `ESRCH` no existe un hilo con ese identificador; `EINVAL` el hilo no es acoplable (*detached*) u otro hilo ya lo está esperando; `EDEADLK` se detectó un interbloqueo (p. ej., un hilo intenta esperarse a sí mismo).
Argumentos	• ***pthread_t tid***: identificador del hilo al que se espera; debe ser un hilo acoplado. • ***void **val_ptr***: dirección donde se copia el valor devuelto por el hilo (`pthread_exit` o `return`); `NULL` si no interesa.
Nota	Solo se puede esperar a un hilo acoplado y solo una vez. También libera los recursos del hilo terminado, como `wait` con los procesos.

Compilación y ejecución de programas con hilos

```
$ gcc -Wall -pthread -o hilo1 hilo1.c
$ ./hilo1
Hilo principal (idthread -> [139780180461376])
Hilo (idthread -> [139780176213696])
```

- **`-pthread`**: indica al compilador que el programa usa hilos POSIX; define las macros necesarias y enlaza con la biblioteca `libpthread`.
- La forma antigua `-lpthread` solo enlaza la biblioteca; `-pthread` es la opción recomendada y portable.
- Incluir siempre <`pthread.h`>. Las funciones devuelven 0 o un código de error (no usan `errno`).
- Los ejemplos imprimen `pthread_self()`, el identificador (`pthread_t`) del hilo que llama, convertido a `unsigned long`.





Ejemplos

Ejemplo 1: hola desde varios hilos (Pacheco)

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

/* Variable global: accesible a todos los hilos */
int thread_count;

void *Hello(void *rank);   /* función del hilo */

int main(int argc, char *argv[]) {
   long thread;  /* long por si el sistema es de 64 bits */
   pthread_t *thread_handles;

   if (argc < 2) return 1;  /* argv[1]: número de hilos */
   thread_count = strtol(argv[1], NULL, 10);
   thread_handles = malloc(thread_count * sizeof(pthread_t));

   for (thread = 0; thread < thread_count; thread++)
      pthread_create(&thread_handles[thread], NULL,
                     Hello, (void *) thread);

   printf("Hello from the main thread\n");

   for (thread = 0; thread < thread_count; thread++)
      pthread_join(thread_handles[thread], NULL);

   free(thread_handles);
   return 0;
}  /* main */
```

```c
void *Hello(void *rank) {
   long my_rank = (long) rank;  /* long por si es de 64 bits */

   printf("Hello from thread %ld of %d\n",
          my_rank, thread_count);
   return NULL;
}  /* Hello */
```

```
$ gcc -Wall -pthread -o hello hello.c
$ ./hello 4
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from the main thread
Hello from thread 2 of 4
Hello from thread 3 of 4
```

Cada hilo recibe su rango (0 a n-1) como argumento, convertido a `void *`: se pasa el **valor**, no un puntero. El orden de los mensajes cambia de una ejecución a otra: lo decide el planificador.

Fuente: Pacheco, An Introduction to Parallel Programming, cap. 4 (pth_hello.c).

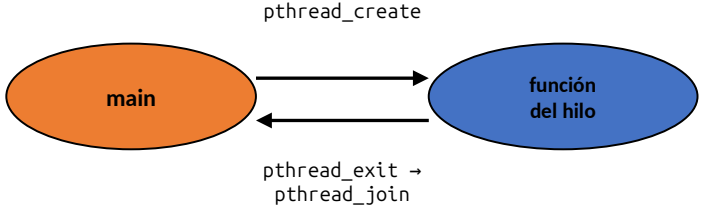
Ejemplo 2: crear un hilo y esperar a que termine

```c
#include <stdio.h>
#include <string.h>
#include <pthread.h>

void *funcion_maneja_hilo(void *);

int main(void) {
    pthread_t pidhilo;
    int r = pthread_create(&pidhilo, NULL,
                           funcion_maneja_hilo, NULL);
    if (r != 0) {   /* r es el código de error */
        fprintf(stderr, "error: %s\n", strerror(r));
        return 1;
    }
    printf("Hilo principal (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    pthread_join(pidhilo, NULL);
    return 0;
}

void *funcion_maneja_hilo(void *param) {
    (void) param;   /* este hilo no recibe argumento */
    printf("Hilo (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    pthread_exit(NULL);
}
```




```
$ ./hilo1
Hilo principal (idthread -> [139780180461376])
Hilo (idthread -> [139780176213696])
```

`pthread_create` devuelve 0 o un **código de error** (no `-1` ni `errno`): se traduce con `strerror(r)`. `main` espera con `pthread_join`: sin esa espera podría terminar antes de que el hilo imprima.

`pthread_t` es un tipo opaco: aquí se convierte a `unsigned long` solo para imprimirlo (válido en Linux/glibc).

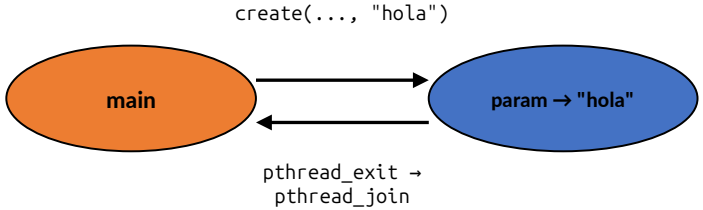
Ejemplo 3: pasar un argumento simple al hilo

```c
/* hilo2.c: envío de un parámetro simple (una cadena)
   al hilo en el momento de crearlo.                    */
#include <stdio.h>
#include <pthread.h>

void *funcion_maneja_hilo(void *);

int main(void) {
    pthread_t pidhilo;
    pthread_create(&pidhilo, NULL, funcion_maneja_hilo,
                   (void *) "hola");
    printf("Hilo principal (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    pthread_join(pidhilo, NULL);
    return 0;
}

void *funcion_maneja_hilo(void *param) {
    printf("Hilo param->%s  (idthread -> [%lu])\n",
           (char *) param, (unsigned long) pthread_self());
    pthread_exit(NULL);
}
```




```
$ ./hilo2
Hilo principal (idthread -> [135971293988672])
Hilo param->hola  (idthread -> [135971291002560])
```

El cuarto argumento de `pthread_create` es un `void *`: aquí, la dirección de la cadena "hola". La función lo recibe como `param` y lo convierte al tipo real con `(char *) param`.

Ejemplo 4: pasar una estructura al hilo

```c
/* hilo3.c: envío de parámetros a un hilo mediante
   una estructura de datos.                          */
#include <stdio.h>
#include <pthread.h>

void *funcion_maneja_hilo(void *);

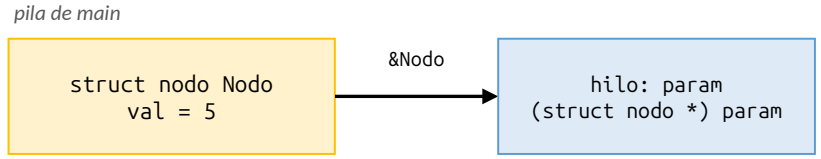
struct nodo {
    int val;
};

int main(void) {
    pthread_t pidhilo;
    struct nodo Nodo;
    Nodo.val = 5;
    pthread_create(&pidhilo, NULL, funcion_maneja_hilo,
                   (void *) &Nodo);
    printf("Hilo principal (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    pthread_join(pidhilo, NULL);
    return 0;
}
```

```c
void *funcion_maneja_hilo(void *param) {
    printf("Hilo param->%d  (idthread -> [%lu])\n",
           ((struct nodo *) param)->val,
           (unsigned long) pthread_self());
    pthread_exit(NULL);
}
```

```
$ ./hilo3
Hilo principal (idthread -> [140702786922304])
Hilo param->5  (idthread -> [140702784681664])
```

Para pasar varios datos se agrupan en una `struct` y se envía su dirección. El hilo la recibe como `void *` y la convierte con `(struct nodo *) param`. La estructura debe seguir existiendo mientras el hilo la use: aquí `main` la espera con `pthread_join`.



Ejemplo 5: un error común al crear varios hilos

```c
/* hilo4_error.c: error al enviar parámetros a varios
   hilos con una única estructura de datos.            */
#include <stdio.h>
#include <pthread.h>

void *funcion_maneja_hilo(void *);

struct nodo {
    int val;
};

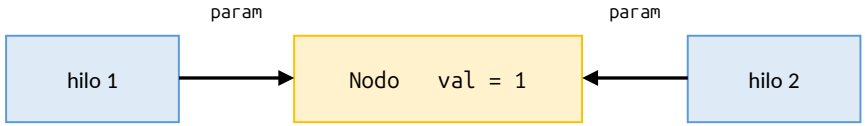
int main(void) {
    pthread_t pidhilo[2];
    struct nodo Nodo;
    int i;

    for (i = 0; i < 2; i++) {
        Nodo.val = i;
        pthread_create(&pidhilo[i], NULL,
                       funcion_maneja_hilo, (void *) &Nodo);
    }
    printf("Hilo principal (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    for (i = 0; i < 2; i++) {
        pthread_join(pidhilo[i], NULL);
    }
    return 0;
}
```

```c
void *funcion_maneja_hilo(void *param) {
    printf("Hilo param->%d  (idthread -> [%lu])\n",
           ((struct nodo *) param)->val,
           (unsigned long) pthread_self());
    pthread_exit(NULL);
}
```

```
$ ./hilo4_error
Hilo param->1  (idthread -> [140704982496960])
Hilo param->1  (idthread -> [140704974104256])
Hilo principal (idthread -> [140704985323328])
```

Error común: los dos hilos reciben la dirección de la **misma** variable Nodo. El hilo principal la modifica en la siguiente iteración (`Nodo.val = 1`) antes de que el primer hilo la lea: ambos imprimen 1 (o cualquier valor, según el orden de ejecución).



Ejemplo 6: una estructura propia para cada hilo

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *funcion_maneja_hilo(void *);

struct nodo {
    int val;
};

int main(void) {
    pthread_t pidhilo[2];
    struct nodo *Nodo;
    int i;

    for (i = 0; i < 2; i++) {
        Nodo = (struct nodo *) malloc(sizeof(struct nodo));
        Nodo->val = i;
        pthread_create(&pidhilo[i], NULL,
                       funcion_maneja_hilo, (void *) Nodo);
    }
    printf("Hilo principal (idthread -> [%lu])\n",
           (unsigned long) pthread_self());
    /* Esperar la terminación de los hilos */
    for (i = 0; i < 2; i++) {
        pthread_join(pidhilo[i], NULL);
    }
    return 0;
}

void *funcion_maneja_hilo(void *param) {
    printf("Hilo param->%d  (idthread -> [%lu])\n",
           ((struct nodo *) param)->val,
           (unsigned long) pthread_self());
    free(param);   /* cada hilo libera su estructura */
    pthread_exit(NULL);
}
```

```
$ ./hilo5_ok
Hilo principal (idthread -> [140176570632000])
Hilo param->0  (idthread -> [140176567301824])
Hilo param->1  (idthread -> [140176558909120])
```



Cada hilo recibe su **propia** estructura, reservada con `malloc` en cada iteración, así el hilo principal no la sobrescribe; el hilo la libera con `free` al terminar. El orden de las líneas depende del planificador.

Resumen

- Un hilo (proceso ligero) es un flujo de ejecución dentro de un proceso: comparte código, datos y archivos con los demás hilos y tiene su propio contador de programa, registros y pila.
- Ventajas: comunicación por memoria común, creación y cambio de contexto baratos, y paralelismo real en sistemas multinúcleo.
- Los hilos pueden gestionarse en el kernel (KLT: el SO los planifica) o en espacio de usuario (ULT: una biblioteca; el kernel ve un solo proceso).
- pthread es la API POSIX de hilos en C: `pthread_create` crea, `pthread_exit` termina, `pthread_join` espera y recoge el valor de salida; se compila con `gcc -pthread`.
- El argumento del hilo es un `void *`: para varios datos se pasa un puntero a una estructura, y cada hilo debe recibir su propia copia.
- Como comparten memoria, los hilos deben sincronizar sus accesos a los datos comunes (tema siguiente).

Referencias y lecturas

- Pacheco, P. S. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011. Capítulo 4: Shared-memory programming with Pthreads (ejemplo pth_hello.c).
- Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*, 10.ª ed. Wiley, 2018. Capítulo 4: Threads & Concurrency (figuras 4.1 a 4.4).
- Carretero, J., García, F., de Miguel, P. y Pérez, F. *Sistemas operativos: una visión aplicada*. McGraw-Hill (figuras de estados y paralelismo de procesos ligeros).
- Tanenbaum, A. S. y Bos, H. *Modern Operating Systems*, 4.ª ed. Pearson, 2015. Sección 2.2: hilos de nivel de usuario y de nivel de kernel.
- Páginas de manual: `man 7 pthreads`, `man 3 pthread_create`, `man 3 pthread_join`, `man 3 pthread_exit`.